EVALUATION SCHEME

Qn. Set Code-7(SOT)

Semester: 5th
Programme: B.Tech
Branch: Comp. Engg.

AUTUMN END SEMESTER EXAMINATION-2025

5th Semester B.Tech

DESIGN AND ANALYSIS OF ALGORITHMS

CS30001 / CS 2012

(For 2023 & Previous Admitted Batches)

Time: 2 Hours 30 Minutes | Full Marks: 50

Answer any FIVE questions.
Question paper consists of two SECTIONS i.e. A and B.
Section A is compulsory.
Attempt any Four question from Sections B.
The figures in the margin indicate full marks.
Candidates are required to give their answers in their own words as far as practicable and all parts of a question should be answered at one place only.

SECTION-A

1. Answer the following questions. [1 × 10]

(a) Which of the following algorithm design paradigms satisfies "Optimal Substructure" property?
i. Greedy, Divide & Conquer
ii. Dynamic Programming, Greedy
iii. Divide and Conquer, Dynamic Programming
iv. All the above

Solution: (ii)
Marking Scheme: 1 for correct choice, 0 for wrong choice.

(b) Which of the algorithm uses the random input to reduce the expected running time or memory usage, but always terminate with a correct result in a bounded amount of time?
i. Las Vegas Algorithm
ii. Monte Carlo Algorithm
iii. Dynamic Algorithm
iv. None of the above

Solution: (i)
Marking Scheme: 1 for correct choice, 0 for wrong choice.

(c) Express in brief with justification what the following algorithm computes for the matrix (A) of size n x n.

Boolean Enigma(A)
FOR $i \leftarrow 1$ to $n-1$ DO
  FOR $j \leftarrow i+1$ to n DO

IF $A[i,j] \neq A[j,i]$ THEN
RETURN $false$
RETURN $true$

Solution: Checks whether the input matrix is a symmetric matrix or not.
Marking Scheme: 1 for correct answer, 0 otherwise.

(d) When a top-down approach of dynamic programming is applied to a problem, it usually _____
   i. Decreases both, the time complexity and the space complexity
   ii. Decreases the time complexity and increases the space complexity
   iii. Increases the time complexity and decreases the space complexity
   iv. Increases both, the time complexity and the space complexity

Solution: Best choice is (iv). However, full credit may be given if someone answers "None of the choices".
Justification: As the time complexity is concerned, for any given problem, the Time Complexity is usually the same for both DP approaches, but the actual execution time will include the constant factor overhead, which is typically higher for the top-down, recursive approach. On the other hand, though the asymptotic space complexity is often the same for both top-down and bottom-up approaches, the top-down method typically has the additional stack space overhead, making its practical memory consumption slightly higher.
Marking Scheme: 1 for correct answer, partial credit as per teachers perception.

(e) Define Matrix Chain Multiplication Problem.

Given matrices A1, A2, …, An, where dimensions are compatible,
the goal is to determine the parenthesization order that minimizes
the total scalar multiplications.

DP Recurrence:
M[i][j] = min over k (M[i][k] + M[k+1][j] + p[i−1] · p[k] · p[j]).

Marking Scheme: 1 for correct answer, partial credit as per teachers perception.

(f) Consider the following functions:
$f(n) = 3n^{\sqrt{n}}$, $g(n) = 2^{\sqrt{n}\log_2 n}$, $h(n) = n!$
Which of the following is/are true?
   i. $h(n) = \Theta(f(n))$
   ii. $h(n) = \Theta(g(n))$
   iii. $f(n) = \Theta(g(n))$
   iv. $g(n) \neq \Theta(f(n))$

Solution: (iii)

Justification: $g(n) = 2^{\sqrt{n}\log_2 n} = \left(2^{\log_2 n}\right)^{\sqrt{n}} = n^{\sqrt{n}}$.

Also, n! grows much faster than $n^{\sqrt{n}}$.

Marking Scheme: 1 for correct choice, 0 for wrong choice.

(g) Let G be an edge-weighted undirected graph with positive edge weights. Suppose a positive constant α is added to the weight of every edge. Which ONE of the following statements is TRUE about the minimum spanning trees (MSTs) and shortest paths (SPs) in G before and after the edge weight update?

i. Every MST remains an MST, and every SP remains an SP.

ii. MSTs need not remain MSTs, and every SP remains an SP.

iii. Every MST remains an MST, and SPs need not remain SPs.

iv. MSTs need not remain MSTs, and SPs need not remain SPs.

Solution: (iii)

Justification: Every Minimum-Cost Spanning Tree (MST) of the original graph remains an MST of the modified graph, because MST algorithms (like Kruskal's or Prim's) depend only on the relative ordering (which edge is smaller than which) of the edge weights, not their absolute values. On the other hand, Shortest Path (SP) between two nodes may change, as adding a constant to every edge weight penalizes longer paths (paths with more edges) more severely than shorter paths (paths with fewer edges).

Marking Scheme: 1 for correct choice, 0 for wrong choice.

(h) Consider the following C-program fragment in which i, j and n are integer variables.

```
i = n;
for (j = 0; i > 0; i /= 2)  j += i;
```

Let val(j) denotes the value stored in the variable j after termination of the for loop. Find the tightest asymptotic bound for the function val(j).

Solution: val(j) = $\Theta(n)$

Justification: val(j) $\approx$ n + n/2 + n/4 + … + 1 = n(1 + 1/2 + 1/4 + … + 1/n) $\leq$ 2n

Marking Scheme: 1 for correct choice with justification. Partial credit as per teacher's discretion.

(i) 05. Consider a decision problems Q. 3CNF–SAT reduces in polynomial time to Q. Then which one of the following is consistent with the above statement?

i. Q is in NP
ii. Q is in NP hard.
iii. Q is in NPC
iv. Q is in P

Solution: (ii)
Marking Scheme: 1 for correct choice, 0 for wrong choice.

(j) Define equal length encoding and how is it different from Hoffman encoding?

Solution: Equal-Length Encoding, also known as Fixed-Length Encoding or Fixed-Length Coding (FLC), is a character encoding scheme where every symbol in the alphabet (e.g., character, pixel value, etc.) is represented by a binary code (codeword) of the exact same bit length. Equal-Length Encoding prioritizes simplicity, speed, and guaranteed unique access. It is highly inefficient for compression when symbol frequencies vary greatly (e.g., in English text, the letter 'e' is far more common than 'z'). Huffman Coding (a type of Variable-Length Coding - VLC), on the other hand, prioritizes minimizing the average codeword length (and thus compression). By using shorter codes for common symbols, the overall length of the encoded message is dramatically reduced.
Marking Scheme: 1 for correct choice, 0 for wrong choice.

SECTION-B

2. (a) An array of integers is said to be UNIMODAL if all its entries are distinct and they are in increasing order until its largest element, after which elements are in decreasing order. Design an algorithm to compute the largest element of a UNIMODAL that runs in O(log n). [6]

2a. Solution:

```
int Largest_Unimodal(a, l, h)
// a: name of the array, l: lower index, h: upper index
{
if (l == h) return(a[l]);          ---------- 1
else
    {
    m = (l + h)/2;          ---------- 1
    if (a[m] < a[m + 1])    ---------- 1
      return(Largest_Unimodal(a, m + 1, h)); ----------T(n/2)
    else
      return(Largest_Unimodal(a, l, m));---------- T(n/2)
     }
}
```

Execution Time:

$T(n) = 1$; for $n == 1$ (for one element) $T(n) = T(n/2) + 1$; for $n>1$

Now, $T(n) = \Theta(\log n)$;

OR

Solution: A unimodal array looks like this:

$a[0] < a[1] < ... < a[k] > a[k+1] > ... > a[n-1]$

The goal is to find the peak index k, where a[k] is the largest element. To find this largest element in a unimodal array in O(log n) time, we can use a binary search on the peak as follows:

```
function find_max_unimodal(A):
   low = 0
   high = n - 1

   while low < high:
      mid = (low + high) // 2

      if A[mid] < A[mid + 1]:
         # We are on the increasing slope ⇒ peak is to the right
         low = mid + 1
      else:
         # We are on the decreasing slope ⇒ peak is at mid or to the left
         high = mid

   # When low == high, we've found the peak
   return A[low]
```

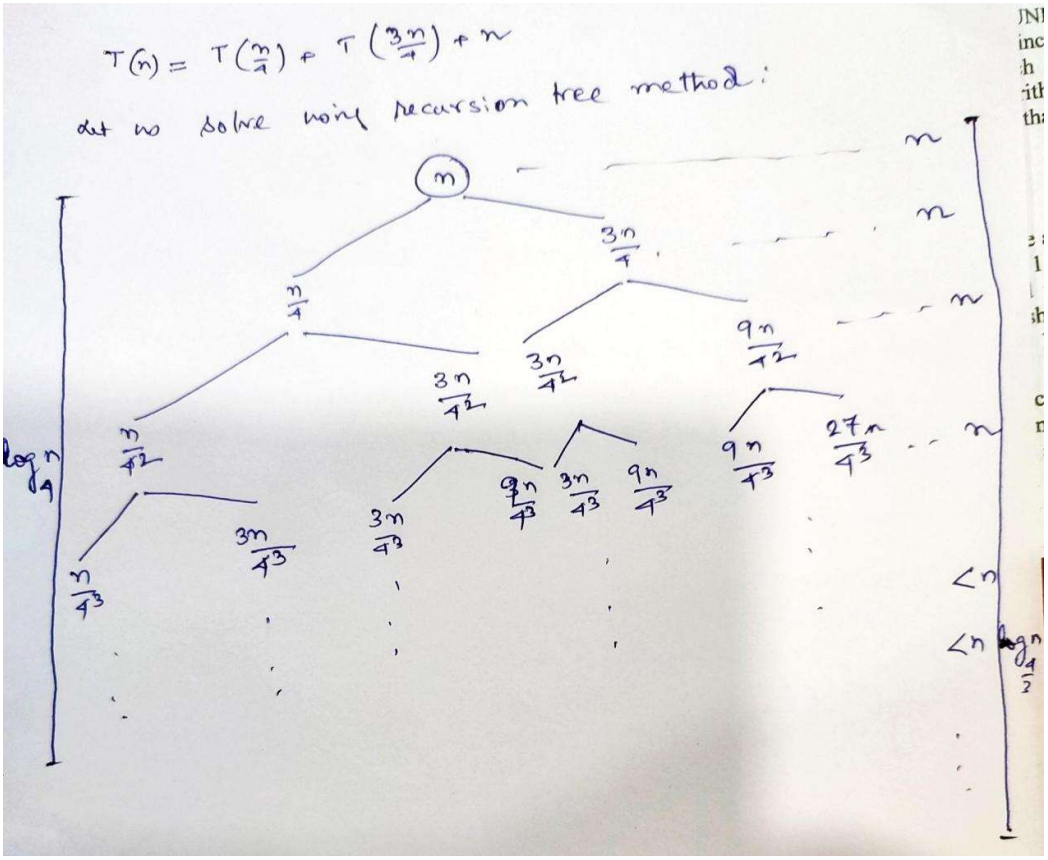
Marking Scheme: 6 marks for correct algorithm. Partial credit as per teacher's discretion.

(b) Solve the following recurrence – [4]

$T(n) = T(n/4) + T(3n/4) + n$; for $n>1$

$T(1) = 1$

Solution: Let us solve this recursion using Recursion Tree method as follows:



Thus every level of the recursion contributes to Θ(n) at the most. The recursion depth equals the number of times the largest piece 3/4 must be multiplied until it drops to constant Θ(1). So, recursion depth for the extreme left branch is $\log_4 n$ and for the extreme right branch is $\log_{4/3} n$, other branches remain within these two limits. Hence total work Θ(n) per level × Θ(logn) levels = Θ(nlogn).

Marking Scheme: 4 marke for correct solution. Partial credit as per teacher's discretion.

3. (a) Given an array, A of size n. Write an algorithm to push one digit number from position 1 to n in order in A. Update all the previously stored numbers from index position i-1 down to 1 while pushing a digit (d) to the index position i ($i=2^k$; k=>1) by 10 x A(p) + d (for p= i-1 down to 1). [6]

Solution: We have an array A[1…n]. We push one-digit numbers sequentially into positions 1 to n. When you push a digit d into index i, if i is of the form 2^k, $k \geq 1$, then we must update all earlier indices: A[p]←10×A[p] + d for p = i−1 down to 1. Otherwise, just store the digit normally.

Algorithm PushDigits(A, n):

```
for i = 1 to n do
    read digit d          // input digit 0–9
    A[i] ← d              // push the digit
```

```
    if (i is a power of 2) then
       for p = i-1 down to 1 do
          A[p] ← 10 * A[p] + d
       end for
    end if
 end for
```

Marking Scheme: 6 marks for correct solution. Partial credit as per teacher's discretion.

(b) Use aggregate analysis or accounting method to determine the amortized running cost per operation (both pushing and update) of the algorithm in 4(a). [4]

3b. Solution:

Let n be 256

The number of push operations for each position is 1;

P: The number of push operations for n position is n x 1 = n;

Q: The number of updates for the pushing into the following positions are as follows:

Position ($i=2^k$; k=>1)	Number of update operations ($i=2^{k-1}$;k=>1)
2	1
4	3
8	7
16	15
32	31
64	63
128	127
256	255

Total number of push and update operations for 256v elements:

T(256) = P + Q;

= 256 + 1 + 3 + 7 + 15 + 31 + 63 + 127 + 255;

= 256 + (2-1) + (4-1) + (8-1) + (16-1) + (32-1) + (64-1) + (128-1) + (256-1);

= 256 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 – 8;

= 256 + 1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 – 8 – 1;

Consider $n = 2^k$; k=>1

T (for pushing and updating n elements) = $n + 2^{k+1} - 1 - k - 1 = n + 2n - 1 - \log n - 1$;

T (for pushing and updating n elements) = O(n);

T (for pushing and updating 1 elements) = O(n/n) = **O(1);**

Marking Scheme: 4 marks for correct solution. Partial credit as per teacher's discretion.

4. (a) Select(a, n) is a function that returns the largest element position in an unsorted array, a from position 1 to n on a non-deterministic machine with running time O(1). Write an algorithm to sort n elements in an array in sorted order and find the running time. [5]

4a. Solution:

```
void Sort(a, n)
// a: name of the array, n: number of elements to be sorted
{
for i = n down to 2 do ---------- n
{
k = Select(a, i); ----------(n - 1) x 1
swap( a(i), a(k) ); ----------(n – 1) x 1
}
}
```

Execution Time:

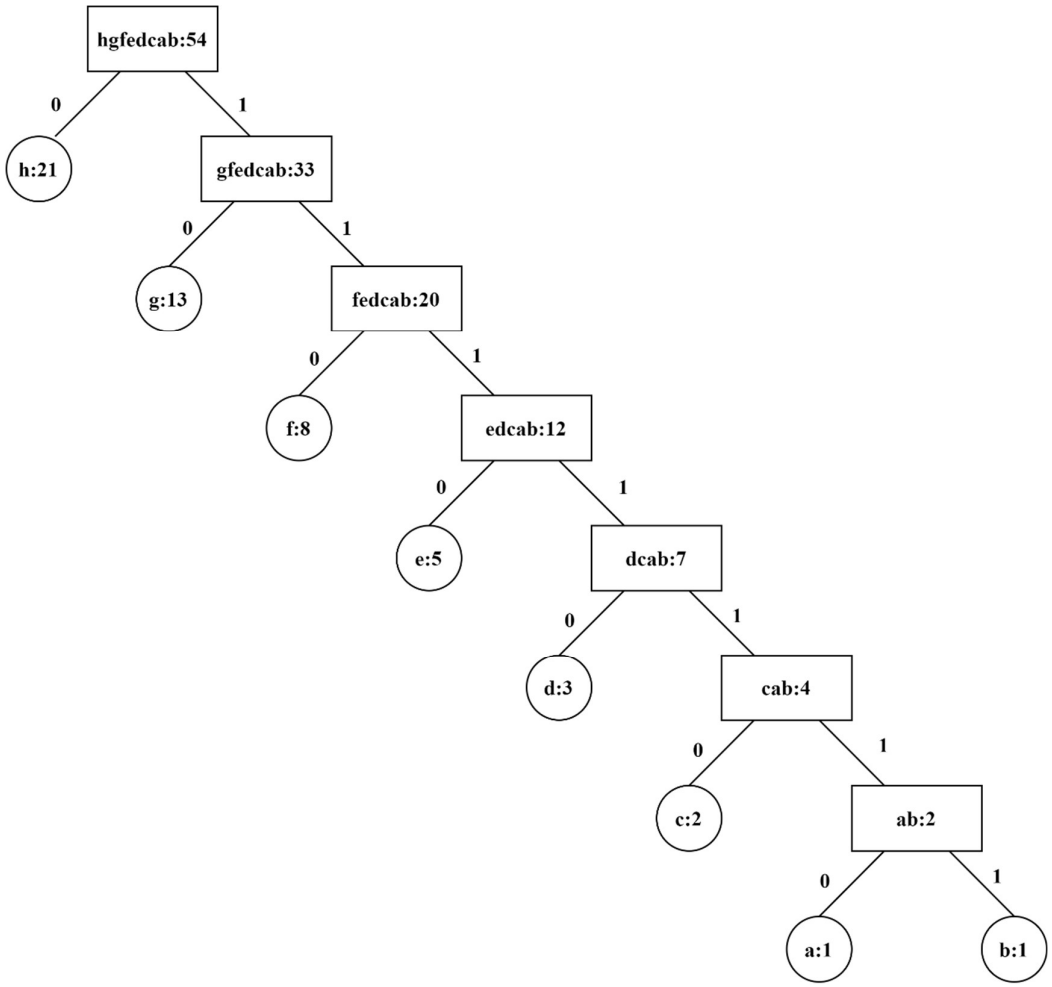
$T(n) = n + n - 1 + n - 1;$

Now, $T(n) = \Theta(n)$;

Marking Scheme: 5 marks for correct solution. Partial credit as per teacher's discretion.

(b) The characters 'a' to 'h' have the following set of frequencies: {a : 1, b : 1, c : 2, d : 3, e : 5, f : 8, g : 13, h : 21}. A Huffman code is used to represent the characters. Find the character string corresponding to the following encoded bit string – 110111100111010, provided that it is a valid encoded text. [5]

Solution:



H=0
G=10
F=110
E=1110
D=11110
C=111110
B=1111110
A=1111111

110	11110	0	1110	10
f	d	h	e	g

Marking Scheme: 5 marks for correct solution. Partial credit as per teacher's discretion.

5. (a) Let $S = \{a, b, c, d, e, f, g\}$ be a collection of objects with benefit-weight values given as follows: a: (12, 4), b: (10, 6), c: (8, 5), d: (11, 7), e: (14, 3), f: (7, 1), g: (9, 6). Derive an optimal solution to the 0/1 knapsack problem for S assuming knapsack can hold objects with total weight 18. [5]

Item	a	b	c	d	e	f	g

Benefit	12	10	8	11	14	7	9
Weight	4	6	5	7	3	1	6

Knapsack Capacity = 18

So, we have the following DP Table:

I/k	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
g	0	0	0	0	0	0	9	9	9	9	9	9	9	9	9	9	9	9	9
f	0	7	7	7	7	7	9	16	16	16	16	16	16	16	16	16	16	16	16
e	0	7	7	14	21	21	21	21	21	23	30	30	30	30	30	30	30	30	30
d	0	7	7	14	21	21	21	21	21	23	30	32	32	32	32	32	34	41	41
c	0	7	7	14	21	21	21	21	22	29	30	32	32	32	32	38	40	41	41
b	0	7	7	14	21	21	21	21	22	29	31	32	32	32	32	39	40	42	42
a	0	7	7	14	21	21	21	26	33	33	33	33	34	41	43	44	44	44	44

An Optimal solution for the above: x = [1, 0, 0, 1, 1, 1, 0]
One may cross check the validity of the solution as follows:
$1 \times 12 + 0 \times 10 + 0 \times 8 + 1 \times 11 + 1 \times 14 + 1 \times 7 + 0 \times 9 = 12 + 11 + 14 + 7 = 44$

Marking Scheme: 5 marks for correct solution. Partial credit as per teacher's discretion.

(b) Suggest the data structure(s) used to store the MCST of a graph in the Kruskal's Algorithm. Write an algorithm to display the edges of the MCST. [5]

5b Solution:
Kruskal Algorithm's Data Structure used to store the edges of the MCST:
Two arrays, V1[n-1] and V2[n-1] of size n-1 where n is the number of vertices

```
Void Display_MCST_edge( V1, V2, n)
{
For I = 1 n – 1 do
{
Print the vertices of the ith edge included in
MCST Print (V1[i], V2[i]);
}
```

OR

Data Structures Used:

1. Edge list sorted by weight
2. Disjoint Set Union (Union-Find)
3. MST Edge List

Algorithm DISPLAY-MST(MST)

1. for each edge (u, v, w) in MST do
2. print u, "–", v, ": weight =", w

OR

Solution: The primary data structure used to store and manage the sets of connected components during the execution of Kruskal's algorithm for finding the Minimum Cost Spanning Tree (MCST) is a **Disjoint Set Union** (DSU) data structure, also often called a **Union-Find** data structure. While DSU manages the sets of vertices, the algorithm's first step requires another data structure to handle the edges of the graph: the graph's edges are typically stored as a list of tuples (e.g., (u, v, cost)). This list must then be sorted by the edge cost in non-decreasing order. Kruskal's algorithm iterates through this sorted list. A A simple **list/array/vector** of selected edges may be used for this purpose. Therefore, the combination of a **sorted Edge List** and a **Union-Find** (DSU) structure is fundamental to Kruskal's algorithm.

Algorithm for displaying edges of Kruskal's MCST:

Algorithm Display-MCST(MCST)
1. Print "Edges of the Minimum Cost Spanning Tree:"
2. For each edge (u, v, w) in MCST do
3. Print "(" u ", " v ") with weight = " w
4. End For
5. Print "Total number of edges in MCST = " length(MCST)
6. End Display-MCST

Marking Scheme: 2 marks for correctly suggesting data structures: UNION-FIND/DSU and Edge List. 3 marks for correctly writing the edge display algorithm. Partial credit as per teacher's discretion.

6. (a) Show the longest common subsequence table, L, for the following two strings: X = "skullandbones", Y = "lullabybabies". Find a longest common subsequence between these strings using a dynamic programming method. [5]

Solution: The DP table for the strings X = "skullandbones" and Y = "lullabybabies":

X/Y	ε	l	u	l	l	a	b	y	b	a	b	i	e	s
ε	0	0	0	0	0	0	0	0	0	0	0	0	0	0
s	0	0	0	0	0	0	0	0	0	0	0	0	0	0
k	0	0	0	0	0	0	0	0	0	0	0	0	0	0
u	0	0	1	1	1	1	1	1	1	1	1	1	1	1
l	0	1	1	2	2	2	2	2	2	2	2	2	2	2
l	0	1	1	2	3	3	3	3	3	3	3	3	3	3
a	0	1	1	2	3	4	4	4	4	4	4	4	4	4
n	0	1	1	2	3	4	4	4	4	4	4	4	4	4
d	0	1	1	2	3	4	4	4	4	4	4	4	4	4
b	0	1	1	2	3	4	5	5	5	5	5	5	5	5
o	0	1	1	2	3	4	5	5	5	5	5	5	5	5
n	0	1	1	2	3	4	5	5	5	5	5	5	5	5
e	0	1	1	2	3	4	5	5	5	5	5	6	6	6
s	0	1	1	2	3	4	5	5	5	5	5	6	6	7

So, LCS(X = "skullandbones", Y = "lullabybabies") = “ullabes”

Marking Scheme: 5 marks for correct solution. Partial credit as per teacher’s discretion.

(b) Define NP with respect to Clique Decision Problem (CDC). Write an algorithm to prove CDC belongs to NP. [5]

Max clique problem: A complete subgraph of a graph is a clique. The maximal clique problem is to determine the size of a largest clique in a graph G.
Clique Decision problem: Is there a clique of size 3 in the graph?

Clique Decision Problem (CDC) is in NP?
We have to verify the certificates of a CDC in polynomial time.

Verification Algorithm:
Let the certificate (probable solution), all the vertices in the complete subgraph, is given in the array (A) of size n. The cost matrix of size n x n represents the graph. If two vertices, i and j are connected then the cost is 1 else 0.

```
Boolean CDC_Verify(A, n, cost)
{
flag=1;  //assume verified OK
```

```
for i = 1 to n-1 do ---------- O(n)
    for j = i + 1 to n do---------- O(n x n)
        If (cost(A[i], A[j] == 0)
        { flag = 0; break; }

if (flag) return
TRUE        else
        return
FALSE
}
```

Execution Time:

$T(n) = \Theta(n^2)$; // Polynomial Time

As the algorithm can be verified in Polynomial Time, CDC belongs to NP.

Marking Scheme: 2 marks for correctly defining NP problems with respect to CDC problem. 3 marks for correctly writing the Polynomial Time Verification algorithm. Partial credit as per teacher's discretion.